PART 12

Integration & Tooling

101 Webhooks

102 API Versioning

103 Semantic Versioning

104 Infrastructure as Code

105 Terraform

106 Helm Charts

107 Build Caching

108 Dependency Hell

#101

Webhooks

WHAT IT IS

HTTP callbacks: a service POSTs to your URL when an event happens.

WHY IT MATTERS

Real-time integration without polling (Stripe, GitHub, Slack).

EXAMPLE

```
Route::post('/webhooks/stripe', function (Request
    $r) {
    $sig = $r->header('Stripe-Signature');
    $event = \Stripe\Webhook::constructEvent(
    $r->getContent(), $sig, config(
    'services.stripe.webhook_secret')); // verify
    signature

    ProcessStripeEvent::dispatch($event->id,
    $event->toArray()); // ack fast, work async
    return response()->noContent();
});
```

#102

API Versioning

WHAT IT IS

Evolving an API without breaking existing clients.

WHY IT MATTERS

Mobile apps and integrators cannot update instantly.

EXAMPLE

```
Route::prefix('v1')->group(base_path(
    'routes/api_v1.php'));
Route::prefix('v2')->group(base_path(
    'routes/api_v2.php'));
```

#103

Semantic Versioning

WHAT IT IS

MAJOR.MINOR.PATCH: major = breaking, minor = backwards-compatible feature, patch = bug fix.

WHY IT MATTERS

Lets consumers judge upgrade risk.

EXAMPLE

1.4.2 → 1.4.3 (fix), 1.5.0 (feature), 2.0.0 (breaking). In Composer: "vendor/pkg": "^1.4" allows >=1.4.0 <2.0.0; ~1.4.2 allows >=1.4.2 <1.5.0. Tag releases: git tag v1.5.0.

#104

Infrastructure as Code

WHAT IT IS

Defining infrastructure (servers, networks, DBs) in version-controlled files.

WHY IT MATTERS

Reproducible environments, code review for infra, easy disaster recovery.

EXAMPLE

Terraform/Pulumi/CloudFormation for cloud; Ansible for configuration; Docker/Compose/Helm for runtime. Workflow: PR → plan reviewed → apply by CI. Avoid manual console changes (drift).

#105

Terraform

WHAT IT IS

A declarative IaC tool: describe desired state, Terraform computes and applies the diff.

WHY IT MATTERS

Multi-cloud, huge provider ecosystem, plan-before-apply safety.

EXAMPLE

```
resource "aws_db_instance" "main" {
  identifier        = "taskflow-prod"
  engine            = "postgres"
  engine_version    = "16"
  instance_class    = "db.t4g.large"
  allocated_storage = 100
  storage_encrypted = true
  multi_az          = true
}
```

#106

Helm Charts

WHAT IT IS

The package manager for Kubernetes: templated manifests plus values.

WHY IT MATTERS

Deploy the same app to dev/staging/prod with different configs; versioned releases and easy rollback.

EXAMPLE

```
# values.yaml
image: { repository: registry.example.com/taskflow,
    tag: "1.4.2" }
replicaCount: 3
ingress: { host: app.taskflow.com }
```

#107

Build Caching

WHAT IT IS

Reusing results of previous builds (layers, dependencies, compiled assets).

WHY IT MATTERS

Faster CI and deploys, lower cost.

EXAMPLE

```
COPY composer.json composer.lock ./
RUN composer install --no-dev --no-scripts   #
    cached until lockfile changes
COPY . .
```

#108

Dependency Hell

WHAT IT IS

Conflicting, outdated or transitive dependency requirements that you cannot satisfy simultaneously.

WHY IT MATTERS

Blocks upgrades, causes security gaps and weird bugs.

EXAMPLE

```
composer why-not laravel/framework 12.0   # who
    blocks the upgrade?
composer outdated --direct
composer audit                             # known
    vulnerabilities
```

Found this useful?

Save it for later.

Share it with a teammate.

Full guide with all code examples:
the Backend Engineering Handbook (GitHub).